



UNIVERSIDAD DE BUENOS AIRES
FACULTAD DE CIENCIAS EXACTAS Y NATURALES
DEPARTAMENTO DE COMPUTACIÓN

Escalando la síntesis composicional de controladores mediante una nueva técnica de minimización para branching bisimilarity

Tesis de Licenciatura en Ciencias de la Computación

Cindy Eliana Levi

Director: Sebastian Uchitel

Codirector: Hernán Gabriel Gagliardi

Buenos Aires, 2026

ESCALANDO LA SÍNTESIS COMPOSICIONAL DE CONTROLADORES MEDIANTE UNA NUEVA TÉCNICA DE MINIMIZACIÓN PARA BRANCHING BISIMILARITY

Esta tesis presenta la definición y análisis exploratorio sobre las fases que están involucradas en el algoritmo *on-the-fly* de síntesis de directores para resolver problemas de Control de Eventos Discretos con la propiedad central de tipo non-blocking. Se busca de esta manera encontrar posibilidades de mejora en la heurística RA, la cual logra actualmente buenos resultados frente a heurísticas tradicionales. Luego de realizada la exploración y análisis inicial sobre el comportamiento de las etapas en la exploración, implementamos algunos cambios sobre la heurística RA para poder experimentar y analizar los resultados. De esa manera realizamos una propuesta de mejora para fortalecer las etapas de exploración analizadas previamente y lograr mejor *performance*.

Palabras claves: Sistemas de Eventos Discretos, Síntesis de Controladores, Control Dirigido, Control Supervisado, Síntesis on-the-fly, LTS, Non-Blocking.

AGRADECIMIENTOS

A Silvana y Beto, por criarme y acompañarme en cada paso con infinito amor.

A Mariana, Laura, Cecilia y Mariano, porque siempre me hicieron sentir querido, protegido y ayudado.

A la casa de Perón, porque en comunidad transcurrió gran parte de la carrera y en donde Milagros, Ale, Julls, Alicia, Franco, Tito, Martín, Cabeza y Virginia me bancaron las jornadas de estudio y siguieron cada uno de mis exámenes.

A Fernando, Pablo, Santiago y Juan Pablo por el ejercicio incesante de la amistad y por ser un ejemplo a seguir dentro y fuera de la facultad.

A Fabio y a Roni, porque hicimos codo a codo cada examen, final y TP de la carrera.

A Daniela, que en muchos momentos difíciles de la carrera supo ayudarme y convencerme de seguir.

Al Departamento de Computación y toda la gente que lo compone por ser una segunda casa desde hace tantos años. Al LaFHIS y todxs sus integrantes por haberme recibido en este último tramo con los brazos abiertos y haberme apoyado en todo momento.

A mi director de Tesis Sebastian Uchitel y a mi co-director Sebastian Zudaire por la guía y por la inmensa generosidad con la que me acompañaron a lo largo de este trabajo.

A la militancia de Identidad, porque aprender de ellxs que vale la pena organizarse, luchar y trabajar por una facultad y un país mejor es una de las cosas más valiosas que me dejaron estos años. A Juliana y Abril, que me permiten irme del claustro con el inmenso orgullo de haber votado y acompañado a las primeras dos Presidentas Peronistas del CECEN.

A Julia porque construimos juntxs, desde el amor, los días más felices.

Índice general

1..	Introducción	1
2..	Antecedentes	5
2.1.	Definiciones utilizadas	5
2.1.1.	Autómatas y composición paralela	5
2.1.2.	Controlador	6
2.2.	Un ejemplo concreto	7
3..	Exploración <i>on-the-fly</i>	11
3.1.	Agnosticismo a la heurística	12
3.2.	Algunas heurísticas utilizadas	12
3.3.	Un ejemplo de ejecución	13
4..	Definición y análisis de las Fases de Exploración	17
4.1.	Explicación general	17
4.2.	Definición de las fases de exploración	17
4.2.1.	Fase 1: encontrar un estado marcado	17
4.2.2.	Fase 2: cerrando un ciclo <i>exitoso</i>	18
4.2.3.	Fase 3: garantizando controlabilidad del ciclo	19
4.2.4.	Fase 4: propagación de estados Goal	21
4.3.	Peso de las etapas en la exploración	24
4.3.1.	Descripción de los casos de estudio	24
4.3.2.	Resultados	25
4.3.3.	Conclusiones	28
5..	Implementación del algoritmo	29
5.1.	Introducción	29
5.2.	Primer desarrollo del algoritmo	29
5.3.	Primeros tests	32
5.4.	Optimizaciones y mejoras (v1)	33
5.4.1.	Estructura del bucle: dos fases que alternan	33
5.4.2.	Fase 1: estabilización de estados	34
5.4.3.	Fase 2: refinamiento de bunches	34
5.4.4.	Construcción del MTS minimizado	34
5.5.	Decisiones de diseño no dictadas por el paper	34
5.5.1.	Dos fases en lugar del nesting outer-while + inner-for	34
5.5.2.	Identidad por referencia para la cola de bunches	34
5.5.3.	Índice inverso estado destino $\rightarrow$ bunches	34
5.5.4.	Subdivisión sistemática de splitters obsoletos	34
5.5.5.	SCCs como unidad atómica en split	34
5.5.6.	Detección de bottom states por SCC	34
5.5.7.	Hashing de bloques en tiempo constante	34
5.5.8.	Restabilización en ambas direcciones	34

5.6.	Adaptaciones al contexto composicional	34
5.6.1.	Estado de error como bloque inicial separado	34
5.6.2.	Self-loops τ controlables	34
5.6.3.	Initiating actions de fluents fuera de τ	34
5.6.4.	Reuso de partición precomputada	34
5.6.5.	Eventos controlables, no-controlables y estados marcados	34
5.7.	Validación de correctitud	34
5.7.1.	Estrategia	34
5.7.2.	Tests automatizados	34
5.7.3.	Casos de borde cubiertos	34
6..	Conclusiones	35
7..	Apéndice	37
7.1.	Medición del peso de cada Fase por heurística	37
7.1.1.	BFS	37
7.1.2.	RA	40

1. INTRODUCCIÓN

*Lo que vieron mis ojos fue simultáneo:
lo que transcribiré, sucesivo,
porque el lenguaje lo es.
Algo, sin embargo, recogeré.
—JLB*

La automatización de tareas ha sido a lo largo de la historia un desafío *motivante* para las Ciencias de la Computación. Desde sus inicios se buscó la manera de automatizar las tareas tediosas o repetitivas. Yendo un poco más lejos, la posibilidad de una computadora que *se programe* a sí misma es abordada por distintas ramas de la computación. Entre ellas se encuentran muchas agrupadas dentro del término *Inteligencia Artificial*. En el trabajo que presentamos nos abocaremos al caso particular de programas que generan **Síntesis Automática de Controladores**.

En este área de estudio buscamos desarrollar código que **sintetice** una solución de un determinado problema de forma **automática** partiendo de ciertas *reglas y objetivos* (o pre/post condiciones) que se proveen y es necesario cumplirlos. De esta manera, lo que devuelve nuestro programa es una estrategia que garantiza alcanzar una solución, si es que existe. Por otro lado, en el caso de que dadas las reglas y los objetivos planteados no exista una solución posible, avisa sobre la inexistencia de la misma. En el caso de garantizar la existencia de una solución, a la estrategia que permite alcanzarla se la conoce con el nombre de **Controlador**.

Este tipo de programas tienen múltiples aplicaciones, entre ellas podemos nombrar el control de aviones para demarcar incendios forestales [4], donde a veces incluso es necesario que el programa pueda adaptarse en medio de su ejecución.

El problema de *síntesis* fue estudiado dentro de distintas áreas como: Control de Eventos Discretos [5], Síntesis Reactiva [6] y Planning automático [7]. Si bien este trabajo se basa en resultados que provienen de combinar enfoques de las tres áreas, nos basamos principalmente en Control de Eventos Discretos (Discrete Event Control o DEC).

En este enfoque modelamos el problema utilizando autómatas finitos, también conocidos como máquinas de estados finitas. Un autómata finito, está conformado por estados y transiciones entre ellos. Podemos pensar los *estados* como puntos y las transiciones como flechas de un solo sentido que unen a uno o varios de ellos. A estos puntos los llamamos estados ya que representan un momento particular del dominio que se quiere modelar, por ejemplo en el caso del avión mencionado anteriormente se podría hablar de estados que representen posibilidades como: *avión aterrizado*, *avión sin combustible*, *avión en vuelo*, etc.

Por otro lado, vamos a tomar dentro de nuestro modelo a las transiciones como posibles acciones. En este caso algunas transiciones las podemos controlar y otras estarán fuera de nuestro alcance. Volviendo al ejemplo del avión, una acción **controlable** podría ser *encendido de motor* mientras que la acción *aterrizaje de emergencia* probablemente sea representada en nuestro modelo como una acción **no controlable**.

En el autómata mencionado además existen estados distinguidos. Por un lado tomaremos un estado como *inicial* y será el estado desde el cual empezaremos la exploración.

Además habrá *estados objetivos* que serán los estados a los cuales buscaremos llegar. En este caso de análisis y a lo largo de la tesis llamaremos *marcados* a estos estados objetivo y buscaremos que el sistema no solo pueda alcanzarlos, sino que sea capaz de hacerlo infinitas veces y de manera segura. Esta última condición de *seguridad* se refiere intuitivamente a que debemos llegar a un estado marcado mediante una secuencia de acciones que evite estados no controlables no deseados en el sistema. A esto se le agrega la condición de llegar al estado marcado infinitas veces, lo que refiere a que buscaremos poder mantenernos *de forma controlable* con transiciones que nos garanticen un ciclo de permanencia en el estado marcado.

El valor de este enfoque y de un algoritmo que permita realizar síntesis es la generalización a distintos problemas. El autómata descrito puede modelar un avión como el caso mencionado, o el funcionamiento de una agencia de viajes como se verá en otro ejemplo o podrá modelar el funcionamiento de diversos problemas complejos. De todas formas, nuestro algoritmo podrá generar un **controlador** sin necesitar ningún cambio específico. Para lograr esto trabajará un experto en el área, quien decide las reglas y objetivos que conformarán el autómata a analizar, por ejemplo partir de un avión en un hangar e intentando llegar a un estado marcado que describa a un avión aterrizado luego de formar un patrón en el cielo. Pero como podemos ver, este experto no debe preocuparse por como se sintetiza el controlador que resuelve el autómata que modela su problema, esa parte está garantizada por el algoritmo de síntesis que genera el controlador.

Por último cabe destacar que el modelado de un sistema complejo resulta prácticamente imposible utilizando un solo autómata. Por esta razón es que se suelen modelar distintas partes del problema original como autómatas independientes y luego se componen para formar el modelo completo del sistema. Esta composición a la que nos referimos debe respetar ciertas reglas y garantiza que un estado del sistema compuesto representa la combinación de estados en que está cada componente al finalizar.

Tradicionalmente la forma de resolver el problema consiste en tomar estos autómatas independientes, realizar la composición y obtener el autómata completo para que sea analizado por el programa para sintetizar el controlador. Sin embargo, cuando el problema a resolver escala en tamaño de estados y transiciones de forma abrupta el autómata compuesto se vuelve demasiado grande, haciendo imposible su análisis y en ocasiones resultando imposible también el proceso de composición.

En este contexto surge el enfoque de exploración *on-the-fly*, en donde la composición se construye a medida que se van explorando transiciones mediante las cuales intentamos resolver el problema de síntesis evitando, en la medida de lo posible, la composición de todo el modelo. Este enfoque fue presentado por primera vez en la tesis doctoral de Daniel Ciolek [1] y luego profundizado en la tesis de Licenciatura de Matías Durán y Florencia Zanollo [2] cuyo trabajo resultó publicado [3].

Basándonos en este trabajo, en esta tesis analizaremos el comportamiento del algoritmo de exploración del autómata compuesto que describimos anteriormente. Vamos a definir y describir ciertas fases o etapas relevantes del proceso, buscando experimentar el desempeño de las heurísticas existentes y que posibilidades de mejora existen en base a esta información.

En el capítulo 2 presentaremos los antecedentes y definiciones necesarias alrededor del problema de síntesis de controladores. Luego en el capítulo 3 presentaremos la idea básica de exploración *on-the-fly* junto con una descripción del enfoque y pseudocódigo asociado como paso necesario para encarar el análisis posterior.

Habiendo presentado las intuiciones del algoritmo, en el capítulo 4 avanzamos en la definición y análisis de las fases de exploración involucradas. En ese capítulo realizamos las definiciones necesarias y hacemos una experimentación acerca del peso de estas fases utilizando la heurística RA definida en [1] y comparándola con heurísticas tradicionales.

En el capítulo 5 avanzamos con una propuesta de cambio a la heurística RA que involucra el aprovechamiento de información brindada en el análisis de etapas del algoritmo y busca mejorar la *performance* de las fases en las que RA tenga menor rendimiento.

Finalmente, en el capítulo 6 presentamos las conclusiones y aportes del trabajo realizado.

2. ANTECEDENTES

En este capítulo agregaremos algunas definiciones y explicaciones necesarias para poder avanzar con la definición y análisis formal del problema composicional de síntesis de controladores.

2.1. Definiciones utilizadas

2.1.1. Autómatas y composición paralela

Definición 1 (*Autómata Determinístico*): Un autómata determinístico es una tupla $T = (S_T, A_T, \rightarrow_T, \bar{t}, M_T)$, donde: S_T es un *conjunto finito de estados*, A_T es un *conjunto finito de eventos del autómata*, $\rightarrow_T: S_T \times A_T \rightarrow S_T$ es una *función de transición*, $\bar{t} \in S_T$ es el *estado inicial del autómata* y $M_T \subseteq S_T$ es el conjunto de *estados marcados*.

Notación 1 (*Pasos y corridas*): Notamos $(t, \ell, t') \in \rightarrow_T$ como $t \xrightarrow{\ell}_T t'$ y lo llamamos un *paso*. A su vez, una *corrida* de una palabra $w = \ell_0, \dots, \ell_k$ en T , es una secuencia de pasos tal que $t_i \xrightarrow{\ell_i}_T t_{i+1}$ para todo $0 \leq i \leq k$, notado como $t_0 \xrightarrow{w} \dots \rightarrow_T t_{k+1}$.

Notación 2: Decimos que un estado s es alcanzado por una palabra w en una corrida comenzando en el estado s' , anotado como $s \in w(s')$, cuando $\exists w_0 \cdot \exists w_1 \cdot w = w_0 \cdot w_1$ y $s' \xrightarrow{w_0} \dots \rightarrow_E s$.

Los autómatas definen un lenguaje, un conjunto de palabras que aceptan. Dado un conjunto de eventos A , notamos con A^* al conjunto de palabras finitas de eventos de A . El lenguaje generado por un autómata T , notado como $\mathcal{L}(T)$, es el conjunto de palabras formadas por eventos que cumplen $\rightarrow_T$. Formalmente, si $w \in A_T^*$, entonces $w \in \mathcal{L}(T)$ si y sólo si existe una corrida para w comenzando desde el estado inicial $\bar{t}$ de T , que notamos $\bar{t} \xrightarrow{w} \dots \rightarrow_T t'$. Generalmente los estados marcados se utilizan para indicar el logro de una tarea. Por lo tanto, consideramos como lenguaje marcado (*o aceptado*) por T (lo cual notamos $\mathcal{L}_m(T)$) al conjunto de palabras cuya corrida termina en un estado marcado.

Formalmente, sea $w \in \mathcal{L}(T)$, entonces $w \in \mathcal{L}_m(T)$ si y solo si hay una corrida de w que comienza en el estado inicial $\bar{t}$ y alcanza un estado marcado $t_m \in M_T$, que notamos $\bar{t} \xrightarrow{w} \dots \rightarrow_T t_m$. Este último concepto es relevante para la definición de la composición paralela. Es una parte fundamental del trabajo para definir las palabras aceptadas por el problema *nonblocking* ya que el lenguaje aceptado por la planta compuesta es el mismo que el aceptado por cada uno de los componentes por separado.

Definición 2 (*Composición Paralela*): La composición paralela ($\parallel$) de dos autómatas T y Q es un operador simétrico y asociativo que produce un autómata $T \parallel Q = (S_T \times S_Q, A_T \cup A_Q, \rightarrow_{T \parallel Q}, \langle \bar{t}, \bar{q} \rangle, M_T \times M_Q)$, donde $\rightarrow_{T \parallel Q}$ es la menor relación que satisface las siguientes reglas (omitimos la versión simétrica de la primera regla):

$$\frac{t \xrightarrow{\ell}_T t'}{\langle t, q \rangle \xrightarrow{\ell}_{T \parallel Q} \langle t', q \rangle} \ell \in A_T \setminus A_Q \qquad \frac{t \xrightarrow{\ell}_T t' \quad q \xrightarrow{\ell}_Q q'}{\langle t, q \rangle \xrightarrow{\ell}_{T \parallel Q} \langle t', q' \rangle} \ell \in A_T \cap A_Q$$

Hay ciertas características de la composición paralela a destacar. En primer lugar, la última regla realiza la sincronización entre componentes. Segundo, que el número de

estados en $T_0 \parallel \dots T_n$ puede crecer exponencialmente con respecto al número de autómatas a componer. Tercero, que el lenguaje aceptado por la composición contiene las palabras que alcanzan estados marcados en todos los componentes simultáneamente.

2.1.2. Controlador

Dado un (conjunto de) autómatas y una partición de sus eventos en dos subconjuntos (controlables y no controlables), lo que buscamos es un *controlador* que limite el vocabulario aceptado por el autómata de forma tal de mantener un camino posible a los estados marcados del mismo.

Un controlador observa las transiciones no controlables y puede deshabilitar algunas transiciones controlables para generar una planta restringida. Una palabra w pertenece al lenguaje generado por T restringido por una función del controlador $\sigma : A_T^* \mapsto 2^{A_T}$ (anotado como $\mathcal{L}^\sigma(T)$) si cada prefijo de w sobrevive a σ . Formalmente, sea $w = \ell_0, \dots, \ell_k$ una palabra en $\mathcal{L}(T)$, entonces $w \in \mathcal{L}^\sigma(T)$ si y solo si para todo $0 \leq i \leq k : \bar{t}^{\ell_0, \dots, \ell_i} \xrightarrow{T} t_{i+1} \wedge \ell_i \in \sigma(\ell_0, \dots, \ell_{i-1})$.

Definición 3 (*Problema de control Safe y Non-Blocking*): Un problema de control con objetivos **Safe** y **Non-Blocking** composicional es una tupla $\mathcal{E} = (E, A_E^C, S_E^I)$, donde E es un conjunto de autómatas $\{E_0, \dots, E_n\}$. En nuestro caso haremos un abuso de notación y usaremos $E = (S_E, A_E, \rightarrow_E, \bar{e}, M_E)$ para referirnos a la composición $E_0 \parallel \dots \parallel E_n$. Por otro lado, $A_E^C \subseteq A_E$ es el conjunto de eventos controlables y definimos análogamente a $A_E^U = A_E \setminus A_E^C$ como el conjunto de eventos **no** controlables.

Una solución para $\mathcal{E}$ es un supervisor $\sigma : A_E^* \mapsto 2^{A_E}$, tal que σ es:

- *Controlable*: $A_E^U \subseteq \sigma(w)$ con $w \in A_E^*$
- *Safe y non-blocking*: para cada palabra $w \in \mathcal{L}^\sigma(E)$ existe una palabra no vacía $w' \in A_E^*$ tal que la concatenación $ww' \in \mathcal{L}^\sigma(E)$ y $\bar{e} \xrightarrow{ww'}_E e_m$ con $e_m \in M_E$ (un estado marcado de E).

Por lo tanto concluimos que un supervisor σ es *controlable* si deshabilita solamente eventos controlables, y es *safe y non-blocking* si podemos garantizar para cualquier palabra del lenguaje una extensión que le permita llegar a un estado marcado. A continuación definiremos como *ganadores* (*perdedores*) a los estados que podemos (no podemos) incluir en un controlador.

Decimos que un estado s es **ganador** en el problema $\mathcal{E} = (E, A_E^C, S_E^I)$ si hay una solución (E_s, A_E^C) donde E_s es el resultado de cambiar al estado inicial de E a s .

Podemos pensar en un controlador *non-blocking* como un jugador optimista que se encarga de no perder, y solo requiere conocer un futuro camino posible para llegar a un estado ganador. Es importante notar que como se busca que cualquier palabra sea extensible a otro estado marcado, lo que se busca es pasar por algún estado marcado infinitas veces. Es decir, un estado e marcado que tenga un camino para que el jugador pueda volver *controlablemente* al mismo estado e .

Por esto es que a lo largo de este trabajo analizaremos detenidamente la estructura de ciclo en nuestro algoritmo ya que sirve para deducir propiedades como que los primeros estados ganadores son aquellos que están en un loop controlable con un estado marcado dentro. Luego anotamos como ganador también a cualquier estado que controlablemente alcanza un estado ganador.

Los ciclos también son esenciales para encontrar los estados perdedores, ya que la única forma de que un estado sea perdedor es que no pueda alcanzar un estado ganador. En otras palabras, los estados perdedores son aquellos que forman parte de un loop que no tiene estados marcados ni transiciones salientes, así mismo los que llevan a un deadlock.

2.2. Un ejemplo concreto

Para ejemplificar y tener una noción acerca del tamaño que puede alcanzar el problema de composición y su posterior exploración, presentaremos una versión simplificada del problema **TravelAgency** que describiremos con mayor detalle posteriormente en el capítulo 4 y será usado para medir la *performance* del algoritmo.

En este problema se busca armar una agencia de venta online de paquetes vacacionales. El objetivo es que la misma orqueste los servicios (alquiler de auto, hotel, pasaje de avión, etc.) de forma automática de manera de obtener un paquete de vacaciones completo de ser posible, evitando pagar por paquetes incompletos.

Para cada servicio que se desea sub-contratar se consulta (query) si ese servicio está disponible. En caso de tener éxito y confirmar la disponibilidad, se espera a confirmar el resto para comprar todos al mismo tiempo. En caso de que algún otro servicio no esté disponible, se cancela la compra y se vuelve al estado inicial, lo cual no implica un gasto.

Este problema, descrito por ahora en términos generales, puede escalar de forma muy rápida si se incrementa la cantidad de servicios a contratar o la cantidad de pasos para reservar cada servicio, por lo que para este ejemplo ilustrativo asumimos que no hacen falta múltiples pasos para reservar un servicio, y solo mostramos el problema con 1 y 2 sub-servicios.

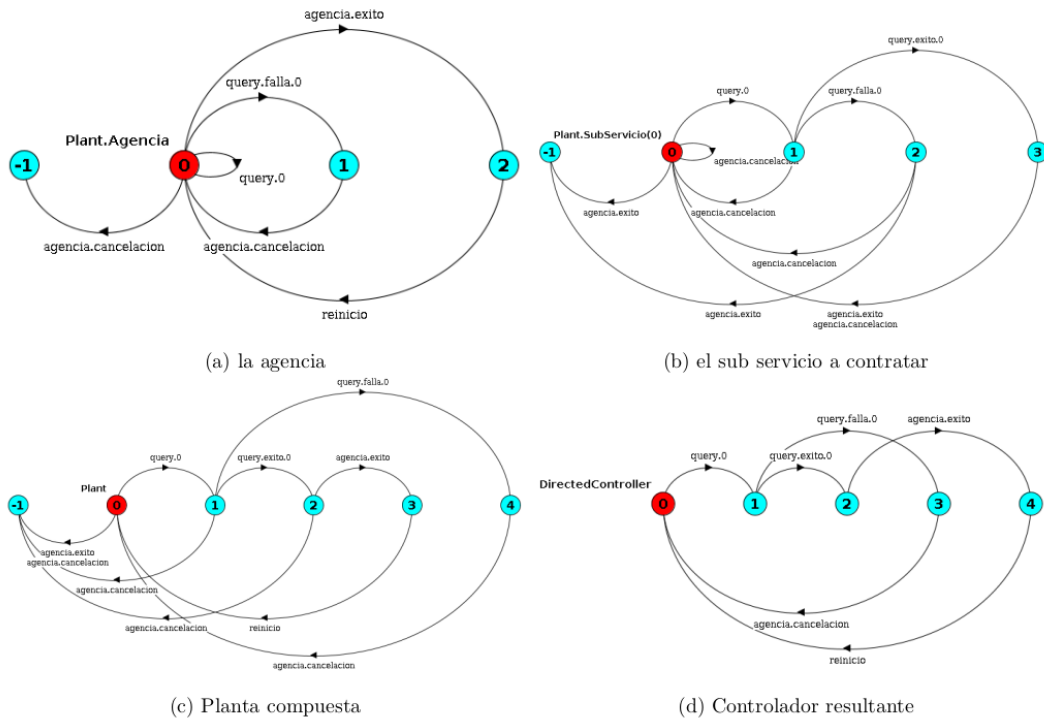


Fig. 2.1: Ejemplo del problema Travel Agency (TA) con un solo sub-servicio

El componente del *subservicio* (figura 2.1b) también tiene el evento *agencia.exito*, pero debe previamente haber recibido la consulta y confirmado su disponibilidad tomando el evento *query.exito.0*.

Podemos observar la representación del sistema complejo en la sincronización que se ve reflejada en la figura 2.1c, que muestra la planta que representa al sistema con los dos componentes previamente descritos.

[illegible]

Fig. 2.2: Planta compuesta para 2 sub-servicios

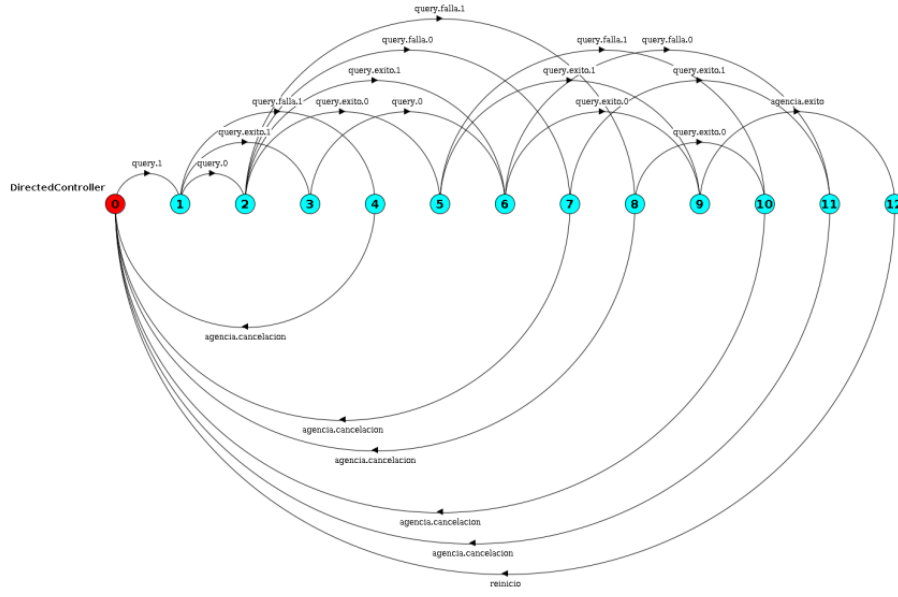


Fig. 2.3: Controlador para el problema que usa 2 sub-servicios

Si analizamos el tamaño que adquiere la planta compuesta expresada en la figura 2.3 podemos notar que es un tipo de problema que escala rápidamente cuando se incrementan los autómatas que se van a componer. También podemos ver que el controlador posee un subconjunto de estados y transiciones de la planta compuesta total.

Por esta razón reafirmamos el enfoque de exploración *on-the-fly* para resolver el problema de síntesis evitando, en la medida de lo posible, la composición de todo el modelo.

3. EXPLORACIÓN *ON-THE-FLY*

El problema de síntesis de controlador tiene una solución clásica que compone toda la planta para luego explorarla. Este enfoque tiene un límite de escalabilidad en el cual la composición de la planta llega al límite de tiempo o memoria y nunca llega a producirse la exploración. Esto se debe al hecho de que al componer distintos autómatas, la cantidad de estados de la composición es exponencial respecto de los estados en los componentes.

De esta forma es que se presenta en [1] la *exploración on-the-fly*, la cual clasifica estados como ganadores o perdedores durante la composición. Haremos una breve presentación del enfoque para que sirva como base para la definición y posterior análisis de las etapas involucradas en el algoritmo de exploración. Para esto primero haremos unas definiciones acerca de lo que significa la exploración *parcial* de la planta.

Definición (*Exploración parcial*): Sea $E = (S_E, A_E, \rightarrow_E, \bar{e}, M_E)$. Decimos que ES es una exploración parcial de E ($ES \subseteq E$) si $S_{ES} \subseteq S_E$ y $ES = (S_{ES}, A_{ES}, \rightarrow_{ES}, \bar{e}, M_E \cap S_{ES})$ donde $\rightarrow_{ES} \subseteq (\rightarrow_E \cap (S_{ES} \times A_E \times S_{ES}))$.

El algoritmo de exploración *on-the-fly* explora incrementalmente el espacio de estados de E logrando una estructura de exploración parcial ES y agregando de a una transición.

Buscaremos de esta forma cortar la exploración de una rama de la planta que ya se sabe perdedora o ganadora, reduciendo así la memoria y tiempo necesarios. Además, si el estado inicial fuera marcado como ganador o perdedor antes de la composición completa de la planta, podríamos obviar lo que resta del proceso de composición.

En el Listing 3.1 mostramos la estructura básica de este método. Trabajando sobre la parte de la planta compuesta hasta el momento (estructura explorada, ES), se expande ES consiguiendo nueva información hasta que sea seguro si el estado inicial es ganador o perdedor en E . Al llegar a esta conclusión se retorna el controlador para $\bar{e}$ en E o se notifica que no es controlable.

Cada una de las partes descriptas generalmente en el Listing anterior tiene muchas variantes posibles que determinan condiciones sobre: cómo se expanden las transiciones, cómo se computan nuevos ganadores y perdedores, etc. En particular, nuestro enfoque consiste en ir agregando una transición a la vez a la parte conocida de la planta, y en cada paso ver si esta nueva transición permite concluir que un estado es ganador o perdedor. Si algún nuevo estado se clasifica como ganador o perdedor, se propaga esta información a sus antecesores, posiblemente marcándolos a su vez como ganadores o perdedores respectivamente.

```

function OTF-Exploration( $E, AE^C$ ):
     $\bar{e} = \langle \bar{e}^0, \dots, \bar{e}^n \rangle$ 
     $Goals = Errors = \emptyset$ 
     $ES = \{\bar{e}\}$ 
    while  $\bar{e} \notin Errors \cup Goals$ :
        ( $e, \ell, e'$ ) = expandNext(heuristic)
         $S_{ES'} = S_{ES} \cup \{e'\}$ 
         $expandES(ES, (e, \ell, e'))$ 

        if  $e' \in Errors$ :
            propagateError( $\{e'\}$ )
        else if  $e' \in Goals$ :
            propagateGoal( $\{e'\}$ )
        else if isLoop( $e, e'$ ):
            if newWinningLoop( $e, e'$ ):
                propagateGoal( $e'$ )
            else if newLosingLoop( $e, e'$ ):
                propagateError( $e'$ )

    if  $\bar{e} \in Goals$ :
        return buildController( $Goals$ )
    else:
        return UNREALIZABLE

```

Listing 3.1: Esquema de algoritmo on-the-fly.

3.1. Agnosticismo a la heurística

Una distinción clave del algoritmo on-the-fly es que está dividido en dos partes. Por un lado se tiene el algoritmo de exploración responsable de que al final se llegue al resultado correcto, por el otro tenemos una heurística que le brinda la próxima transición a explorar.

Ese algoritmo de exploración no depende de la heurística ya que la misma no garantiza siempre elegir el mejor camino posible, sino solo la mejor aproximación que encuentre. Este aspecto está demostrado y comentado ampliamente en [2]. El foco del presente trabajo estará puesto en entender las fases por la que transcurre el algoritmo DCS y el peso que tienen en las distintas heurísticas.

3.2. Algunas heurísticas utilizadas

Si bien mencionamos que el algoritmo es agnóstico a la heurística, nos parece pertinente realizar algunos comentarios al respecto de las heurísticas que utilizaremos en las siguientes secciones para el análisis y experimentación. Por un lado nos referimos a la heurística llamada *BFS* cuyo comportamiento es el clásico en la exploración de estructuras como grafos o autómatas.

Por otro lado, también nos vamos a referir a la heurística *RA* (Ready Abstraction) presentada ampliamente en [1] y [3]. Esta heurística viene inducida por una abstracción que hacemos sobre la planta que nos permite estimar el número de pasos requeridos para alcanzar un estado marcado (desde cualquier otro estado). La abstracción representa una versión reducida del espacio total de estados que deshecha la suficiente información como para evitar el problema mencionado anteriormente como *explosión de estados*, pero a la vez preserva los detalles suficientes como para hacer una buena estimación. Por lo tanto, podemos notar que el objetivo de la heurística está relacionado con conocer, dado un estado que componemos de la planta, la estimación de *cuan cerca* se encuentra de un estado marcado.

3.3. Un ejemplo de ejecución

Para visualizar de forma clara el funcionamiento y las ventajas de la exploración on-the-fly, se muestra a continuación una ejecución posible sobre una planta ejemplo.

Además vamos a establecer una convención que nos permita interpretar este tipo de gráficas con las que representamos a los LTS. Esto lo haremos mediante un código de colores que nos servirán para diferenciar algunas características de cada estado representado con un círculo con una etiqueta para nombrarlo adentro. Describiremos a continuación el código mediante el cual se representarán los siguientes aspectos para calificar estados y transiciones:

- Los estados **coloreados en violeta** y contienen una **M** en su interior identifican a los que consideramos como **marcados** en la planta.
- El estado **coloreado en lila** y que contiene una **I** en su interior representa al estado inicial del cual parte la exploración.
- Los estados **coloreados en un celeste claro** representan estados que ya fueron explorados mediante una transición.
- Los estados y las transiciones **coloreadas en amarillo** representan estados y transiciones que no fueron exploradas.
- Las flechas punteadas representan **transiciones no controlables** entre estados.
- Las flechas sólidas representan **transiciones controlables** entre estados.

Usaremos estas convenciones a lo largo del trabajo aclarando y ampliando oportunamente las referencias cuando haya que destacar aspectos adicionales sobre los estados o las transiciones.

Volviendo al ejemplo que queremos proponer, suponemos que dados ciertos LTS calculamos su composición y llegamos a la planta de la figura 3.1 en la cual plantearemos una exploración donde las transiciones se exploran en orden creciente en cuanto al número que tienen definido (c1, u2, u3, c4, etc).

Al iniciar el algoritmo no veremos la planta completa sino que partimos desde el estado inicial *e0*, desde el cual expandimos la primer transición, en este caso **c1**. Como llega a un estado sin explorar (*e1*) y este estado no representa un estado *deadlock*, no hay información para procesar, por lo cual tenemos que seguir explorando. Incluso si llegamos a un estado explorado (no *deadlock*) no va a haber nueva información si no tenemos un loop. La intuición será que no va a haber nuevos ganadores hasta que se pueda llegar a un marcado infinitas veces, por ende necesita como mínimo que haya un ciclo. Entonces hasta ahora llevamos explorado lo mostrado en la figura 3.2.

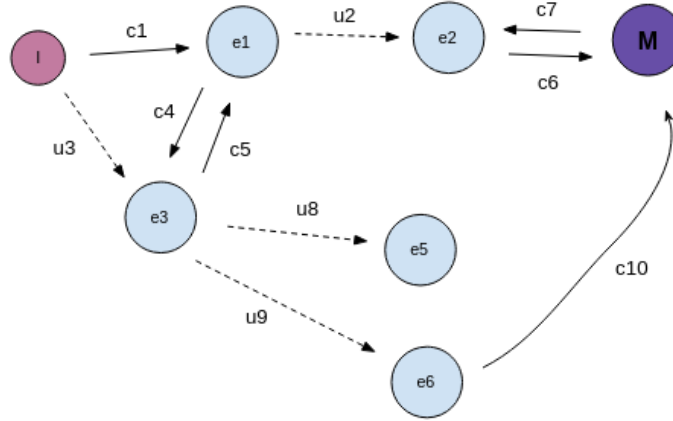


Fig. 3.1: Ejemplo de planta compuesta completa

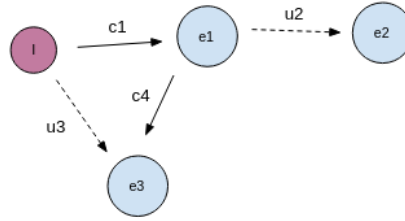


Fig. 3.2: Primeros pasos de la exploración

La exploración continúa por la transición **C5** mediante la cual cerramos un loop [**e1**, **e3**], pero el estado **e3** puede ser forzado a algo no explorado usando la transición no controlable **u8**, entonces todavía no podemos sacar una conclusión. El estado de la exploración hasta ese momento se puede ver reflejado en la figura 3.3.

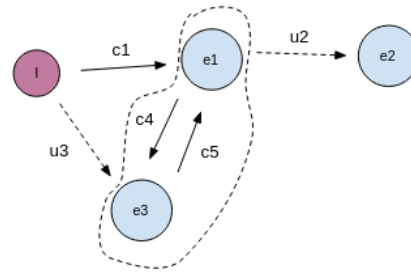


Fig. 3.3: Momento de la exploración donde se encuentra un ciclo sin conclusiones

Si continuamos explorando por la transición **c6** y llegamos al estado marcado **M**, podríamos decir que es ganador pero necesitamos siempre poder llegar a este estado marcado, por ende seguimos sin tener nueva información hasta explorar la transición **c7** me-

diante la cual se cierra el ciclo $[e2, M]$. Este ciclo tiene un estado marcado (estado M) y el estado $e2$ no puede ser forzado a transicionar mediante una transición no controlable, ya que el resto de sus transiciones son controlables; por ende obtenemos nuestros primeros estados ganadores ($e2$ y M). A continuación debemos propagar esta nueva información llegando a marcar al estado $e1$ como *ganador*, porque puede *controlablemente* llegar al ciclo $[e2, M]$. De todas formas todavía no hay conclusión sobre el estado inicial debido a que puede ser forzado a transicionar a un estado sin explorar. Este momento de la exploración se ve reflejado en la figura 3.4.

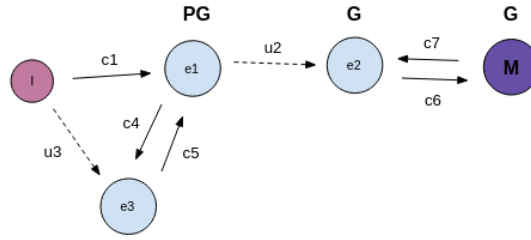


Fig. 3.4: Se encuentran estados ganadores (G) y se propaga (PG)

Proseguimos con la transición $u8$ y llegamos a explorar el estado $e5$, que representa un *estado deadlock*, por ende es etiquetado como *error* directamente.

Propagando esta información marcamos los estados $e3$ y I como errores, obteniendo información concluyente sobre el estado inicial. De esta manera podemos ver claramente que ya no es necesario seguir explorando y el algoritmo nos informa que no existe controlador para la planta. El estado de exploración final de la planta se ve en la figura 3.5.

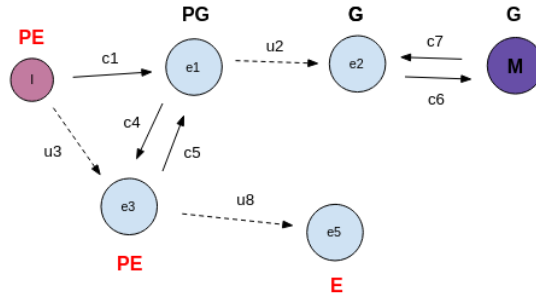


Fig. 3.5: Se encuentra Error(E) y propaga(EP) hasta inicial dejando de explorar

Notar que mediante esta exploración no analizamos las demás transiciones de los estados $e2$ ni $e3$ ya que, en el caso de $e2$ *gana* por la rama que exploramos y el resto son transiciones controlables, y en el caso de $e3$ *pierde* por una transición no controlable.

Es decir que $e2$ puede *ganar controlablemente* y $e3$ es forzado a perder sin importar la forma del resto de la planta. Si bien el algoritmo es agnóstico a la heurística, podemos ver que la decisión de que transiciones exploramos están relacionadas con la heurística que elijamos para guiar la exploración. Este último aspecto será analizado en los siguientes capítulos.

4. DEFINICIÓN Y ANÁLISIS DE LAS FASES DE EXPLORACIÓN

4.1. Explicación general

Durante el proceso de síntesis del controlador que realizamos mediante la exploración on-the-fly de la planta notamos que se atraviesan ciertas etapas (o fases) cuyos objetivos parciales no siempre coinciden, más allá de ser etapas que conducen a un objetivo en común (resolver el problema de control en general). La exploración atraviesa estas etapas de forma totalmente agnóstica a la heurística elegida porque están relacionadas directamente con la estructura del algoritmo de exploración dirigida.

4.2. Definición de las fases de exploración

Vamos a definir cuatro fases que creemos importantes considerar a la hora de elegir una heurística para el problema de control general. Los puntos claves que las definirán son: encontrar el primer estado marcado, cerrar un ciclo *exitoso*, concluir que el ciclo encontrado previamente sea controlable y finalmente la propagación hasta el estado inicial de la condición de controlabilidad.

4.2.1. Fase 1: encontrar un estado marcado

Consideramos como parte de la *Fase 1* de exploración a todas las transiciones y estados expandidos antes de realizar la primera transición cuyo destino sea un estado marcado. Tomamos esa definición porque hasta no encontrar un primer estado marcado en nuestra exploración, por definición tampoco podríamos definir estados *ganadores* como **Goal** entre los explorados. En principio, nuestro enfoque realiza una exploración que parte desde un estado inicial y busca encontrar un ciclo que satisfaga la condición **canBe WinningLoop**. Como vimos anteriormente, la condición exige que el ciclo que evaluamos contenga un estado marcado o que algún estado dentro del mismo pueda alcanzar en un paso a otro estado catalogado como **Goal**. En ese sentido definimos formalmente a la **Fase 1** de exploración como el lapso hasta que expandamos una transición hacia un estado marcado por primera vez.

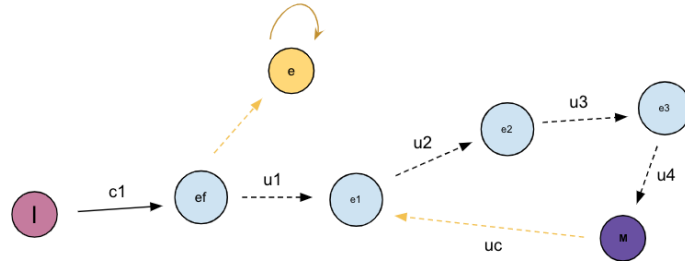


Fig. 4.1: Ejemplo de exploración que finaliza la Fase 1

En la figura 4.1 se puede ver como se requirieron explorar **5 estados** a través de **5 transiciones** llamadas *c1*, *u1*, *u2*, *u3* y *u4*.

4.2.2. Fase 2: cerrando un ciclo *exitoso*

Luego de haber expandido una transición que llegue a un estado marcado y habiendo completado la **Fase 1** que fue descripta anteriormente, buscaremos que se cierre un ciclo sobre ese estado marcado que exploramos. Como podemos ver en el pseudocódigo del algoritmo principal, la función requiere expandir una transición que cierre un ciclo dentro del total de estados explorados. Lo que buscamos será verificar formalmente la noción de cierre de ciclo. Para eso, si expandimos la transición (e, e') , en principio queremos chequear la siguiente condición: $\exists w . e' \xrightarrow{w}_{ES'} e$.

Es decir aseguramos la existencia de un camino w mediante el cual alcancemos e desde e' . En la línea siguiente vamos a conseguir, mediante la función auxiliar **getMaxLoop**, un conjunto con todos los estados que forman parte del ciclo cuya existencia aseguramos en el paso anterior.

Una vez que hayamos chequeado afirmativa esa condición, necesitamos hacer una segunda verificación acerca de ese ciclo en donde nos aseguramos parcialmente ciertas condiciones adicionales que en caso no cumplirse nos obligarían a descartarlo.

Esta segunda verificación se encuentra formalizada como función auxiliar en nuestro pseudocódigo con el nombre de **canBeWinningLoop**, función que toma un conjunto de estados que forman un ciclo y chequea la posibilidad de alguna de estas dos condiciones: que exista un estado marcado dentro de ese ciclo, o que exista algún estado dentro del ciclo evaluado que pueda alcanzar en un paso a un estado etiquetado como *Goal* dentro de la exploración. En nuestro caso, como estamos ejecutando un punto del algoritmo DCS donde aún no hay estados *Goals* dentro de la exploración, tendremos que verificar la primera condición.

En el caso de verificar exitosamente la condición expresada en la función **canBeWinningLoop** de nuestro pseudocódigo habremos encontrado un ciclo potencialmente exitoso y concluimos la **Fase 2** ejecutando, por primera vez en el algoritmo DCS, la función **findNewGoalsIn** para ese mismo ciclo.

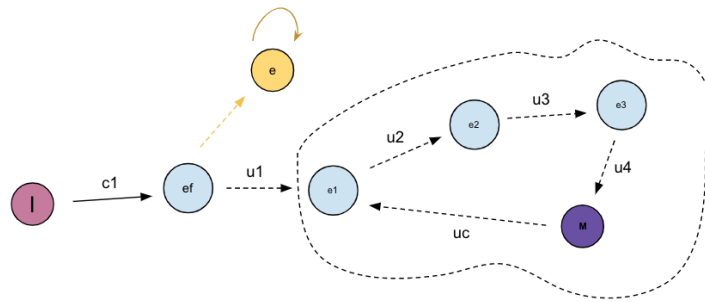


Fig. 4.2: Ejemplo de exploración que finaliza la Fase 2

En la figura 4.2 podemos ver que exploramos una transición adicional que no habíamos transitado en la figura 4.1, la transición no controlable etiquetada como **uc**. Esta transición podemos ver que cierra un ciclo entre los estados $e1$, $e2$, $e3$ y M , pero además es un ciclo *exitoso* que garantiza la condición pedida por el método **canBeWinningLoop** ya que M es un estado marcado dentro del ciclo.

4.2.3. Fase 3: garantizando controlabilidad del ciclo

Esta fase comienza con la primera ejecución de la función **findNewGoalsIn** descrita en el pseudocódigo y usando como parámetro el ciclo que encontramos en la fase anterior. Esta función aplica un método de punto fijo que permite encontrar, en caso de que exista, un subciclo controlable a partir del ciclo original. Aunque los detalles y la formalización de la función están explicados en el pseudocódigo, es importante mencionar que para garantizar la *controlabilidad* del ciclo se aplica iterativamente un proceso que quita algunos estados del conjunto que compone el ciclo original con cierto criterio. La idea será remover iterativamente estados que *rompan* la controlabilidad del ciclo y buscar el conjunto de estados resultantes al momento en que, ante una nueva corrida, no se puedan eliminar más estados del conjunto.

El criterio mencionado para remover iterativamente estados del conjunto original se basa en cumplir alguna de las siguientes dos condiciones sobre cada estado $s \in C$, siendo C el ciclo analizado:

1. $\exists e . forcedTo(s, e, ES'_{\perp}) \wedge e \notin (C \cup Goals)$
2. $\nexists s' \in C, \exists \ell . s' \xrightarrow{\ell}_{ES'} s$

df El punto 1 formaliza la primera razón para *remove* elementos del ciclo analizado durante la ejecución de **findNewGoalsIn**. La fórmula expresa que no puede haber ningún estado dentro del ciclo que esté forzado a irse mediante una transición no controlable a un estado e que no esté en el ciclo ni haya sido previamente marcado como *Goal*.

El punto 2 expresa la condición de ser un estado no alcanzable por ningún otro estado dentro del ciclo. En la primera iteración del método ningún estado cumplirá esta condición ya que el parámetro (un conjunto de estados que conforma un ciclo) conforma un grafo conexo. Una vez que empecemos a quitar elementos de este conjunto por cumplir la condición expresada en punto 1 aparecerán ciertos estados no alcanzables desde ningún otro estado del ciclo, es decir que cumplen la condición número 2.

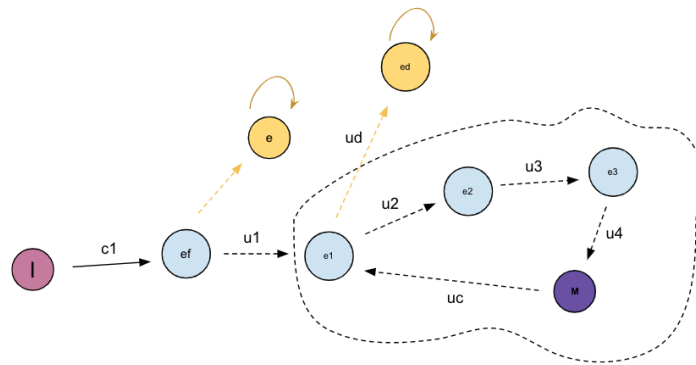


Fig. 4.3: Ejemplo de un ciclo (rodeado por línea de puntos)

En la figura 4.3 podemos ver un ejemplo en donde la línea punteada encierra un ciclo que contiene un estado marcado y cuyos estados que lo componen serán parámetro de la función **findNewGoals**. Luego, al ejecutarse esta función y analizar los estados que componen el ciclo, encontrará que el estado $e1$ es un estado que cumple la primera de las

condiciones que enumeramos anteriormente, ya que posee una transición no controlable (**ud**) hacia un estado ed que no se encuentra dentro del ciclo ni está dentro del conjunto de estados marcados como *Goal*. En ese sentido, el estado $e1$ será removido como parte de la primera iteración del método **findNewGoals**, dando lugar a una estructura reflejada en la figura 4.4.

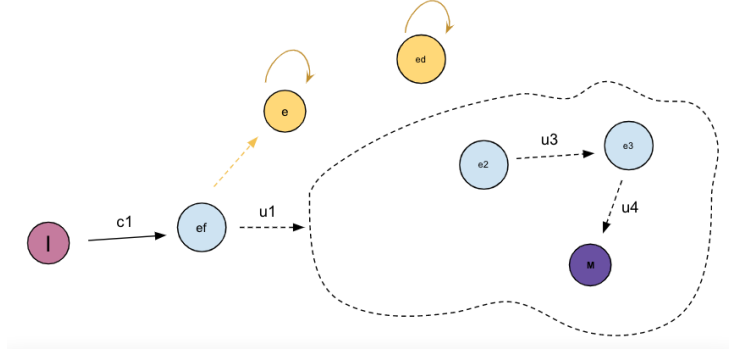


Fig. 4.4: Ejemplo del ciclo luego de remover en una iteración los estados del ciclo que cumplían la condición 1

Lo que podemos ver en la figura 4.4 es el conjunto resultado de la primera iteración del método **findNewGoals** luego de eliminar el estado que mencionamos anteriormente y que destacamos en la figura 4.3. De esta manera destacamos al estado $e2$ que se puede ver como no cumple la condición número dos de las enumeradas previamente porque justamente se trata de un estado que, en esta iteración, ni siquiera recibe transiciones entrantes. Por lo tanto podemos determinar que no será alcanzable desde ningún estado, lo que implica que tampoco será alcanzable por un estado del ciclo. Para el ejemplo citado en las figuras, el algoritmo de punto fijo sobre el conjunto que representa al ciclo resultará con un conjunto vacío ya que todos los estados serán removidos por alguna de las condiciones comentadas previamente. Podemos ver en el pseudocódigo correspondiente a **findNewGoals** que en ese caso devolvería un conjunto vacío que no genera ningún cambio a nivel general de la ejecución del algoritmo DCS. Justamente lo que buscamos en esta fase es lograr un subciclo controlable como resultado de nuestro método, en donde exista una iteración que ya no modifique el conjunto y a la vez haya elementos que compongan el subciclo buscado. En este sentido conviene analizar en mayor profundidad las razones por las cuales el resultado de la ejecución del método **findNewGoals** no resulta exitosa (es decir devuelve un conjunto vacío) y eso lo haremos, en principio, caracterizando algunos tipos de estados que se desprenden de la lógica del método iterativo. En principio analizaremos los estados del ciclo que es parámetro del método **findNewGoals** que cumplen con la condición de estar *forzados a irse* y que formalizamos previamente. La idea de este análisis será ver que características nos permiten agrupar a esos estados. Por un lado, nos interesa analizar los estados forzados a irse **en la primera iteración** del método **findNewGoals**. Es importante chequear esta condición en la primera iteración ya que después el método funciona quitando elementos del conjunto, por lo cual en posteriores iteraciones se pueden encontrar transiciones no exploradas por haber removido previamente al destinatario de alguna de esas transiciones. Estos estados que el método quita en la primera iteración del conjunto son los que desencadenan el resultado negativo sobre el chequeo de controlabilidad

del ciclo. Podemos afirmar que, en caso de *resolver* las transiciones de esos estados que *lo sacan* del ciclo, hubieramos encontrado un subciclo controlable como resultado del método **findNewGoals**.

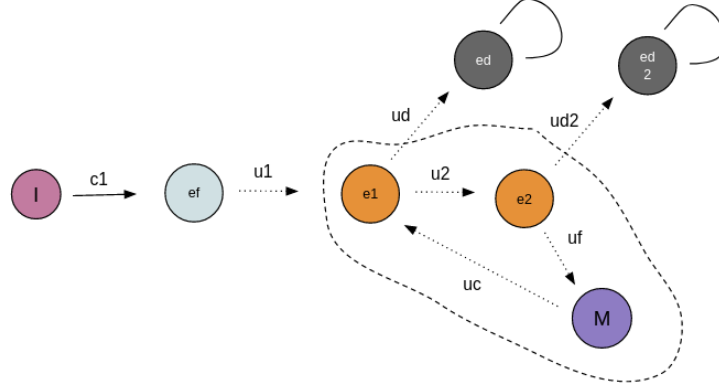


Fig. 4.5: Ejemplo del ciclo evaluado en la primera iteración del método FindNewGoalsIn

En la figura 4.5 y dentro del círculo punteado representamos el estado del ciclo que será evaluado en la primera iteración del método **findNewGoalsIn**. Destacamos dentro del ciclo a los estados **e1** y **e2** ya que ambos poseen transiciones no controlables (*ud* y *ud2* respectivamente) que conducen a estados que están por fuera del ciclo y que no están catalogados como Goal, de hecho son estados Deadlock. Queremos analizar puntualmente a los estados que cumplen esta condición ya que son claves para la resolución de esta fase de exploración. La importancia de estos estados está justificada en el hecho de suponer -para el ejemplo de la figura 4.5- que si las transiciones **ud** y **ud2** condujeran a caminos controlables que *vuelvan* al ciclo entonces resolveríamos el problema de control planteado en **findNewGoalsIn**. La Fase 3 finaliza cuando el problema de controlabilidad del ciclo sea resuelto y encontremos un subciclo controlable, el cual nos servirá de parámetro para empezar la Fase 4.

4.2.4. Fase 4: propagación de estados Goal

Concluimos la fase anterior devolviendo un subciclo controlable que contiene un estado marcado. Este proceso es condición necesaria para resolver nuestro problema en general ya que nos permite lograr no solo alcanzabilidad a un estado marcado, sino también una manera de llegar de forma controlable a el. Por como fue definido el problema de control previamente, partimos la exploración desde un estado inicial desde el cual tenemos que poder alcanzar, también controlablemente, el subciclo mencionado. La Fase 4 inicia con la primera ejecución del método **propagateGoal**, cuyo parámetro será el ciclo controlable que encontramos en la fase anterior. Este método buscará garantizar que podamos llegar controlablemente desde el estado inicial hacia alguno de los estados del ciclo. Por lo tanto, primero buscaremos encontrar todos los estados *ancestros* explorados que no hayan sido etiquetados como *Goal* o *Error* de los elementos del ciclo mediante el método **ancestors-None**. Los *ancestros* conseguidos mediante esta función se formalizan con el siguiente conjunto:

$$\{e \in ES' \mid \exists e' \in ciclo . \exists w . e \xrightarrow{w}_{ES'} e' \wedge \nexists s \in w(e) . s \neq e' \wedge s \in Goals \cup Errors\}$$

Sobre este nuevo conjunto de estados *ancestros* del ciclo controlable y etiquetados como **None**, realizaremos un proceso iterativo similar al de la fase anterior en donde vamos a remover estados que cumplan con algunas de las siguientes condiciones:

- Quitaremos estados que en el proceso de propagación encontremos que estén *forzados a irse*. Esto quiere decir que son estados que tienen transiciones no controlables hacia elementos que no están en el ciclo ni hayan sido etiquetados como Goal previamente. Estos estados se pueden describir formalmente para todos los estados $s \in ancestorsNone(estadosCiclo)$ de la siguiente manera:

$$\mathbf{forcedTo}(s, S_{ES'_\perp} \setminus (C \cup Goals), ES'_\perp)$$

- Por otro lado también vamos a remover iterativamente los estados que no puedan alcanzar un estado **Goal** y cuya condición se describe formalmente en la siguiente línea cuya definición más extensa encontraremos en el pseudocódigo:

$$\neg \mathbf{canReachGoalIn}(s, C, ES'_\perp)$$

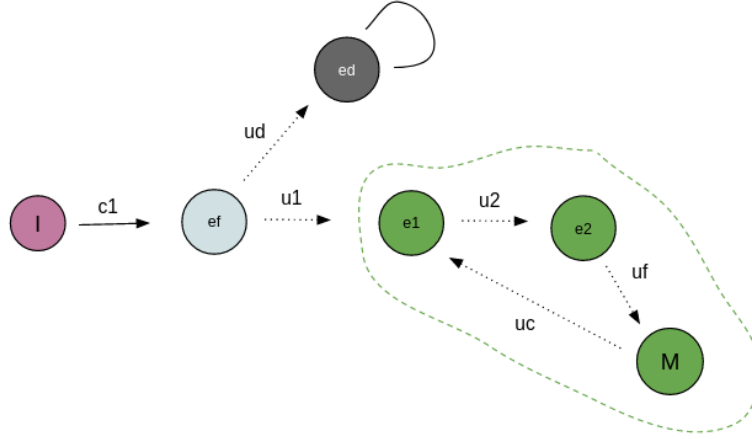


Fig. 4.6: Ejemplo de proceso de PropagateGoal fallido

En la figura 4.6 podemos ver un caso en donde ejecutamos el método **propagateGoal** habiendo completado la Fase 3 por encontrar un subciclo controlable con un estado marcado (marcado con línea de puntos resaltada en verde). Sin embargo, el proceso de propagación iniciado desde todos los estados del ciclo falla al llegar al estado *ef*, el cual tiene una transición no controlable *ef* saliente hacia un estado **marcado como deadlock**. En ese sentido buscamos caracterizar los estados que nos llevan a *perder* en el proceso de la **Fase 4** por no lograr una propagación controlable hacia el estado inicial. Este análisis nos puede dar algunos indicios de cara a la elaboración de una estrategia que minimice las ejecuciones de **propagateGoal**. En principio no encontramos características en las

heurísticas con las que venimos trabajando (*Random*, *BFS*, *RA*) que utilicen esta información de cara a elegir transiciones para la exploración. Este aspecto nos motivará de cara a la siguiente sección, en la cual buscaremos variantes de las heurísticas existentes que permitan utilizar nueva información.

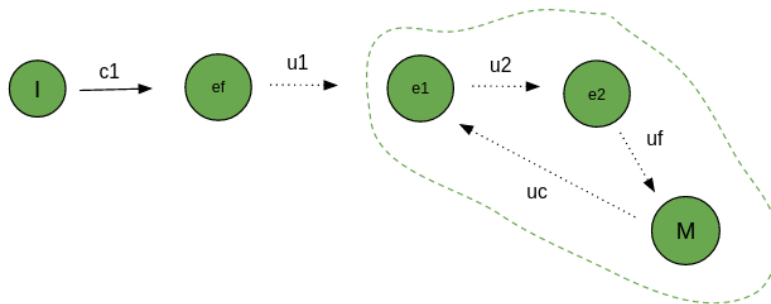


Fig. 4.7: Ejemplo de proceso de PropagateGoal exitoso

En la figura 4.7 podemos ver en cambio un caso en donde concluye exitosamente la Fase 4. El ciclo controlable es idéntico al planteado en la figura 4.6, al igual que el principio de la propagación que busca hacer esta fase. La diferencia está en haber sacado la transición no controlable etiquetada como *ud* que incluíamos en la figura 4.6 y que convertía en no controlable el camino que recorriamos en la propagación desde el ciclo hasta el inicial.

4.3. Peso de las etapas en la exploración

4.3.1. Descripción de los casos de estudio

En la siguiente *subsección* mostramos los resultados de una experimentación que nos permitirán medir el peso relativo que tiene cada una de las etapas descriptas anteriormente en el proceso de exploración general. Decidimos usar el mismo conjunto de problemas que el utilizado para examinar la performance del algoritmo original de síntesis en [2]. A continuación describimos brevemente todos los casos de estudios, los cuales fueron escritos con la posibilidad de ser escalados en el número de componentes y estados:

- **Transfer Line:** Automatización de una fábrica, un dominio de mucho interés en el área de supervisory control. **TL** consiste de n máquinas conectadas por n buffers cada uno con capacidad de k unidades, termina en una máquina adicional llamada Test Unit.
- **Dinning Philosophers:** Problema clásico de concurrencia. En **DP** hay n filósofos sentados en una mesa redonda, cada uno comparte un tenedor con sus vecinos alejados. El objetivo del sistema es controlar el acceso a los tenedores de manera que los filósofos puedan alternar entre comer y pensar; evitando deadlock y starvation. Adicionalmente, cada filósofo, luego de tomar un tenedor, debe cumplir con k pasos de etiqueta antes de comer.
- **Cat and Mouse:** Juego de dos jugadores donde cada uno toma turnos para moverse a una casilla adyacente dentro de un mapa de la forma de un corredor dividido en $2k + 1$ áreas. En **CM** n gatos y la misma cantidad de ratones son colocados en extremos opuestos del corredor. El objetivo es mover a los ratones de manera que no terminen en el mismo lugar que un gato. Los movimientos de los gatos no son controlables. En el centro del corredor hay un agujero que lleva a los ratones a un área segura.
- **Bidding Workflow:** Modela el proceso de evaluación de proyectos de una empresa. El proyecto debe ser aprobado por n equipos. El objetivo es sintetizar un flujo de trabajo que intente llegar a un consenso, es decir, aprobar/rechazar el proyecto cuando todos los equipos lo aceptan/rechazan. La propuesta puede ser reasignada para reevaluación por un equipo hasta k veces, no se puede reasignar si el equipo ya lo había aceptado. Cuando un equipo lo rechaza k veces el proyecto puede ser rechazado sin consenso. Es un caso de estudio típico del dominio de Business Process Management.
- **Air-Traffic Management:** Representa la torre de control de un aeropuerto, que recibe n peticiones de aterrizaje simultáneas. La torre necesita avisar si tiene permiso para aterrizar o, en caso contrario, en cuál de los k espacios aéreos debe realizar maniobras de espera. El objetivo es que todos los aviones puedan aterrizar de manera segura. El problema solo tiene solución si la cantidad de aviones es menor a la de espacios aéreos ($n < k$).
- **Travel Agency:** Modela una página on-line de ventas de paquetes de viajes. El sistema depende de n servicios de terceros para realizar las reservas (ej. alquiler de auto, compra de pasajes, etc). Los protocolos para utilizar los servicios pueden

variar de manera no controlable; una variante es la selección de hasta k atributos (ej. destino del vuelo, clase y fechas). El objetivo del sistema es orquestar los servicios de manera de obtener un paquete de vacaciones completo de ser posible, evitando pagar por paquetes incompletos.

Por otro lado, para medir la performance del benchmark descripto utilizamos el registro de tiempo total, transiciones expandidas y estados expandidos al finalizar la exploración. Todos estos resultados los estaremos midiendo, finalmente, para dos heurísticas que nos permitan realizar una comparación exploratoria sobre su impacto en las fases. Estas dos heurísticas que guían la exploración serán Breadth First Search (BFS) y RA [1]. Descartamos la utilización en este caso de una heurística Random debido al elevado numero de ejecuciones requeridas para tener resultados con estadística ya que las ejecuciones no son deterministas, y debido a que BFS y RA superan en *performance* a Random (podemos verlo en [2], por lo cual esas heurísticas nos alcanzan como base.

4.3.2. Resultados

Sabemos que cada heurística tiene distintos pesos relativos sobre cada una de las fases comentadas como se puede ver en las tablas que figuran en el anexo de este trabajo. Para poder comparar el peso relativo de cada una contrastaremos los dos tipos de heurísticas más exitosas según el trabajo visto en [1] que son RA y BFS.

En la tabla 4.1 podemos ver, para cada caso de estudio, el porcentaje de instancias en las que la heurística RA explora menos estados que la heurística BFS. Consideramos únicamente las instancias que son resueltas por ambas heurísticas garantizando soluciones controlables al problema de síntesis.

Performance de estados explorados por fase RA vs BFS						
Fase / Caso	TA	AT	BW	CM	DP	TL
Fase 1	33 %	30 %	80 %	100 %	100 %	100 %
Fase 2	33 %	25 %	84 %	100 %	100 %	100 %
Fase 3	100 %	85 %	84 %	100 %	100 %	100 %
Fase 4	100 %	25 %	100 %	65 %	100 %	100 %
Total	100 %	85 %	84 %	100 %	100 %	100 %

Tab. 4.1: Porcentaje de instancias que RA resuelve con menor cantidad de estados explorados que BFS por fase

Los resultados que podemos ver en la tabla 4.1 muestran que, considerando el total de estados expandidos, la heurística RA *gana* sobre BFS en mas de un 84 % de las instancias analizadas para cada tipo de caso de estudio. Sin embargo, analizando por fases hay algunos casos donde RA pierde frente a BFS como podemos ver en Fase 1 y Fase 2 de TA y AT, y Fase 4 de AT.

Sin embargo, la tabla 4.1 no nos brinda información cualitativa sobre en qué medida la heurística **RA** mejora en cantidad de estados explorados a la resolución que propone **BFS**. Para eso, proponemos analizar los datos utilizando un nuevo indicador. Definimos para una fase $f \in \{1, 2, 3, 4\}$ fija y dos heurísticas RA y BFS, el indicador I_f de la siguiente manera:

$$I_f = \frac{\sum_{i \in Inst.} CantEstados_{RA}^f(i)}{\sum_{i \in Inst.} CantEstados_{RA}^f(i) + \sum_{i \in Inst.} CantEstados_{BFS}^f(i)}$$

Si consideramos que $CantEstados_{RA}^f(i)$ y $CantEstados_{BFS}^f(i)$ son funciones auxiliares que nos devuelven la cantidad de estados explorados en cada fase f y para una instancia i en ambas heurísticas. De esta forma, sumando y luego comparando el desempeño en el total de las instancias nos otorga una métrica con la que medimos cualitativamente *cuanto mejor desempeño* tiene la heurística RA sobre BFS por fase.

A continuación otorgamos una descripción más precisa acerca del significado que tienen los posibles valores de este indicador y que los podemos interpretar de la siguiente manera:

$$I_f \cong \begin{cases} 0 & \text{si} & \mathbf{RA} \text{ no explora estados en esa fase} \\ 0 < I_f < 0,5 & \text{si} & \mathbf{RA} \text{ explora } \mathbf{menos} \text{ estados que } \mathbf{BFS} \text{ en esa fase} \\ 0,5 & \text{si} & \mathbf{RA} \text{ explora } \mathbf{igual} \text{ cantidad de estados que } \mathbf{BFS} \text{ en la fase} \\ 0,5 < I_f < 1 & \text{si} & \mathbf{RA} \text{ explora } \mathbf{más} \text{ estados que } \mathbf{BFS} \text{ en esa fase} \\ 1 & \text{si} & \mathbf{BFS} \text{ no explora estados en esa fase} \end{cases}$$

Luego de haber definido este nuevo indicador, veremos en la tabla 4.2 los resultados de esta métrica expresada para todos los casos y diferenciada por fase. Particularmente en los casos donde **ninguna de las dos heurísticas** utilice ningún estado para resolver una fase pondremos un guión como referencia en la tabla.

Performance de estados explorados por fase RA vs BFS						
Fase / Caso	TA	AT	BW	CM	DP	TL
Fase 1	0,63	0,32	0,27	0,08	0,01	0,01
Fase 2	0	0,007	0	0,01	0	0
Fase 3	0,27	0,69	0,51	0,09	0,004	0,001
Fase 4	-	-	-	1	0,06	-
Total	0,28	0,42	0,42	0,2	0,007	0,006

Tab. 4.2: Indicador I_f definido para cada caso del Benchmark y Fase definida

En principio conviene remarcar de la tabla 4.2 como *RA* supera en *performance* a *BFS* en todos los casos obteniendo valores cercanos a cero para la etapa 2 y reduciendo los tiempos del resto de las etapas, salvando excepciones como la fase 1 de **TA** y las fases 3 de **BW** y **AT**, donde tenemos valores superiores al 0,5. Particularmente los casos **DP** y **TL** obtienen, usando *RA*, resultados significativamente mejores en cada una de sus fases.

Si bien observamos que en la **fase 1** para el caso **TA** *BFS* logra una mejor performance y en los casos **AT** y **BW** la ventaja de *RA* no es tan significativa, nos gustaría poder ver en cual etapa hay un espacio de mejora. Si bien la tabla 4.2 nos muestra la comparación

de performance entre las dos heurísticas, no nos indica el costo de la fase en el total del algoritmo. Para eso analizaremos, para cada caso de estudio, el porcentaje de estados explorados por fase para la heurística RA. De esta forma tendremos una intuición acerca del peso relativo que tiene la ejecución de cada fase con determinada heurística.

Porcentaje de cada fase medido mediante la acumulación de estados explorados para RA						
Fase / Caso	TA	AT	BW	CM	DP	TL
Fase 1	5 %	56,3 %	17,5 %	2 %	84 %	93 %
Fase 2	0 %	0,4 %	0 %	0,3 %	0 %	0 %
Fase 3	95 %	43,3 %	82,5 %	34 %	14 %	7 %
Fase 4	0 %	0 %	0 %	63,7 %	2 %	0 %
Total	100 %	100 %	100 %	100 %	100 %	100 %

Tab. 4.3: Porcentaje de cada fase medido en estados explorados para la heurística RA

En la tabla 4.3 encontramos, para cada caso de estudio, el porcentaje de estados explorados por fase usando la heurística RA. En la sección de **apéndice** encontraremos los gráficos en los que se mide el peso de cada etapa tanto para RA como podemos ver en la tabla, como para la heurística BFS.

La tabla 4.3 nos proporciona una visión acerca de las fases más *costosas* para resolver el problema de síntesis usando RA. Sin embargo, aunque este aspecto es valioso, es información que deberíamos complementar con la información anterior para buscar un nuevo indicador que nos de una idea de ventanas de mejora dentro de la experimentación realizada. Para eso planteamos una multiplicación de los valores de ambas tablas, con la idea de obtener un valor que combine ambos aspectos.

En este caso, un número alto implica que determinada combinación de fase y caso de estudio obtienen poca ventaja comparativa compitiendo contra BFS (el valor de la tabla 4.2 y además eran originalmente difíciles de resolver para RA porque ocupaban gran porcentaje de los estados explorados en total (el valor en la tabla 4.3).

Indicador de costo de las fases						
Fase / Caso	TA	AT	BW	CM	DP	TL
Fase 1	3,15	18,01	4,72	0,16	0,84	0,93
Fase 2	0	0,003	0	0,003	0	0
Fase 3	25,65	29,87	42,07	3,06	0,056	0,007
Fase 4	0	0	0	63,70	0,12	0

Tab. 4.4: Multiplicación entre los valores de la tabla 4.2 y la tabla 4.3

En la tabla 4.4 se muestra explícitamente que la ventana de mejora para los problemas que le resultan *más difíciles* (TA, AT, CM y BW) a la heurística **RA** se encuentra mayoritariamente en **fase 3** para TA, AT y BW y en **fase 4** para CM. Justamente los valores grandes en la tabla corresponden a que son problemas en donde la fase 3 era originalmente pesada de resolver para RA pero además tiene poca mejora sobre BFS (lo que corresponde a un valor de porcentaje alto en la tabla 4.2).

Por lo tanto, podemos ver luego de estos análisis que existe una gran ventana de mejora a la heurística **RA** en su desempeño de la tercera etapa para la mayoría de los

casos, sobresaliendo **TA**, **AT**, **BW** y **CM** en distinta escala.

Por último mencionamos el caso de **CM** donde podemos ver particularmente el problema de fase 4 expresado en períodos donde ejecuta repetidas veces el método **propagateGoal** sin éxito. Comparando este aspecto con los resultados obtenidos usando las heurísticas vemos que, aún logrando métricas totales parecidas, en **RA** se traslada el peso de la etapa 3 a la etapa 4. Esto indica que, en algunos casos particulares del problema **CM**, la heurística **RA** aporta a resolver rápido la fase 3 pero le cuesta terminar de propagar los estados **Goal** hasta que incluyan al inicial.

4.3.3. Conclusiones

En el análisis de resultados repasamos como el aporte de la **heurística RA** mejora la métrica del total de estados explorados para todos los casos. Sin embargo, si analizamos detalladamente, en los casos donde la mejora es menor comparativamente se puede ver que hay espacio de mejora en la fase 3 que, según el análisis que concluye en la tabla 4.4, es la que mayor cantidad de estados ocupa en resolver el problema en general.

Por lo tanto sostenemos la hipótesis de la posibilidad de mejora para la tercera etapa de exploración por lo comentado anteriormente y considerando que, más allá de la estructura del problema, la heurística **RA** no utiliza información específica que lleve a resolver esa fase.

A continuación haremos un análisis más detallado del comportamiento de la tercera etapa utilizando **RA** para encontrar elementos que nos permitan mejorar las métricas de exploración para esa fase.

5. IMPLEMENTACIÓN DEL ALGORITMO

5.1. Introduccion

La herramienta MTSA [8] es el framework de desarrollo realizado en Java para varias iniciativas de LaFHIS en el contexto de la investigación sobre LTSs. Es por eso que tanto el algoritmo composicional presentado por Uchitel y Gagliardi «AGREGAR CITA» como la versión de WSOE utilizada para la composicionalidad fueron implementados en esta herramienta. Era entonces natural realizar el desarrollo del algoritmo de Branching Equivalence sobre el mismo framework. A su vez, nos permitirá comparar de forma más verídica ambos algoritmos, realizando pruebas de Benchmarks tanto para tiempos de ejecución como para el uso de memoria.

En primer lugar, se realizó la implementación del algoritmo de Branching Equivalence en el archivo BranchingEquivalence.java, creado en mtstools/DCS/Compositional/ ya que bajo este módulo están las herramientas utilizadas por el algoritmo composicional.

A su vez, en CompositionalApproach.java se reemplazó la llamada original al algoritmo de WSOE por la llamada a la nueva función que utiliza el algoritmo de Groote y Jansen, reemplazando también los preprocesamientos necesarios para los algoritmos que se hacen en algunos casos fuera de las funciones principales.

La llamada al algoritmo de Branching Equivalence queda entonces de esta forma

```
Pair<List<Set<Long>>, List<Set<Triple<Long, String, Long>>>> branchingPartition
    = BranchingEquivalence.getPartitions(localControllerMTS,
        ↪ localAlphabet, totalTranslator, goal.getFluents());

MTS<Long, String> minimizationResult =
    ↪ BranchingEquivalence.buildMinimisedMTSFromPartition(localControllerMTS,
        ↪ tauLabels, translatorControllable, branchingPartition);
```

donde en la función getPartitions(...) se implementa el algoritmo de Groote y Jansen y se devuelve la partición final de estados y transiciones, y en buildMinimisedMTSFromPartition(...) se construye el MTS minimizado a partir de la partición calculada.

A su vez, el algoritmo recibe como parámetros el MTS a minimizar, el alfabeto local, el traductor total y los fluents objetivo.

En primer lugar, la modalidad de desarrollo fue implementar la lógica del algoritmo lo más fielmente al paper posible, es decir, sin tomar en cuenta las optimizaciones ni las cotas de complejidad. En la versión cero (v0), el algoritmo replica la estructura del paper pero reemplazando las estructuras de datos propuestas por estructuras nativas de MTSA.

5.2. Primer desarrollo del algoritmo

Como fue explicado en el capítulo 3, el algoritmo de Groote y Jansen mantiene dos estructuras de refinamiento: una partición de estados Π_s y una partición de transiciones Π_t . En Π_s se encuentran los "bloques" de estados estables tal que si dos estados están en distintos bloques entonces NO son branching equivalent. En Π_t se encuentran los "bunches" de transiciones no-inertes, es decir, las transiciones que pueden distinguir estados. Ambas estructuras se refinan a la par: cada vez que un bloque de estados se parte en dos, se actualizan los bunches para reflejar el nuevo bloque, y cada vez que un bunch se

parte en slices por (acción, bloque destino), se generan nuevos splitters para estabilizar los bloques afectados. El proceso continúa hasta llegar al punto fijo donde no quedan ni splitters pendientes ni bunches por refinar: las clases de equivalencia son entonces los bloques finales de Π_s .

La idea para esta primera implementación fue mantener la estructura del paper, donde el loop principal itera sobre cada conjunto de transiciones no triviales que están en Π_t , tomando un subconjunto chico de transiciones. Este subconjunto de transiciones, es separado de su bunch original y añadido a Π_t como un nuevo bunch. Luego, se deben refinar los bloques de Π_s para que queden estables respecto al nuevo conjunto de transiciones obtenido.

Una decisión importante para esta primera versión fue la elección de las estructuras de datos a utilizar, ya que inciden en la complejidad final de la implementación. En esta primera versión, buscando en primer lugar comprender la lógica en sí del algoritmo y poder verificar su correctitud, se eligieron estructuras de datos simples y nativas, lo más similar posible a las del paper.

Π_s se implementó como `List<Set<Long>>`, es decir una lista de conjuntos de estados. Cada `Set<Long>` representa un bloque de estados. Π_t se implementó como `List<Set<Triple<Long, ↪ String, Long>>>`, es decir una lista de conjuntos de transiciones. Cada `Set<Triple<Long, ↪ String, Long>>` representa un bunch de transiciones no-inertes.

A su vez, se debió utilizar una estructura `Pi_tCola` para poder iterar sobre los bunches a la vez que se actualizaba la lista de pendientes a refinar. Tal como se menciona en el paper de Groote y Jansen, se utiliza también la estructura del `splitterList` para almacenar los splitters generados durante el proceso de refinamiento. A su vez, se creó la clase `Splitter` para representar cada splitter, que contiene el bloque que se va a partir, el conjunto de transiciones que cruzan bloques y un flag de primary/secondary, un conjunto de transiciones marcadas para identificar que esos bloques son estables y un `groupId` para identificar qué Splitters se originaron a partir de la misma división inicial.

En primer lugar, el primer preprocesamiento requerido para este algoritmo es eliminar las componentes tau fuertemente conexas. La implementación de este paso usa la lógica del algoritmo de Kosaraju, que tiene una complejidad $O(n + m)$, por lo que cumple con la complejidad prometida por Groote y Jansen. Luego, se debe inicializar Π_s , que arranca con dos bloques (B_{vis} y B_{invis}), donde B_{vis} contiene el conjunto de estados desde los cuales se puede alcanzar una transición visible y B_{invis} es su complemento. Para realizar esta inicialización, se ejecuta la función `computeBvis` que genera un grafo de predecesores, almacenando para cada transición, cuál fue su origen. Luego, identifica cuáles son visibles y realiza un DFS para propagar las acciones visibles e identificar desde qué estados son alcanzables. Por lo tanto, la complejidad resultante de la función es también $O(n + m)$. Finalmente, B_{vis} se inicializa con el conjunto de estados identificados por esta función como visibles y B_{invis} se inicializa con el resto. Π_t es inicializado con un único bunch que contiene todas las transiciones no-inertes.

Una de las primeras estructuras que también se deben inicializar en el preprocesamiento es el `stateToBlockMap`, definido como un mapa `Map<Long, Set<Long>>`, que almacena para cada estado a qué bloque de Π_s pertenece. Esta estructura es indispensable para poder realizar el seguimiento y las modificaciones rápidas que identifican a cada estado con su bloque, ya que la lógica principal del algoritmo se basa en modificar estos bloques continuamente. A su vez, esta estructura es acompañada por la función `updateStateToBlockMap(Map<Long, Set<Long>> map, ↪ Set<Long> newBlock1, Set<Long> newBlock2)` que recibe dos bloques nuevos y actualiza el mapa

para que cada estado de cada bloque nuevo quede asociado a su bloque correspondiente.

Luego del preprocesamiento y la inicialización de las estructuras de datos, el código tiene un loop principal que itera sobre los bunches. En cada ciclo, se toma un bunch T . Si T es un bunch trivial, es decir que todas sus transiciones con la misma acción a van a un mismo bloque, entonces el bunch T no debe ser refinado. Si T no es trivial, entonces se buscan todos los "slices" de T , es decir, se buscan y se separan los subconjuntos de T que tienen la misma acción a y bloque de destino B' . Luego, se itera sobre todos los slices de T y se separa el primer slice encontrado que cumpla que es chico", se lo identifica almacenándolo en la estructura `chosenTransitions`. Ahora, lo que queremos hacer es dividir a T , separando las `chosenTransitions` de T y dejando el resto de T en otra estructura que llamamos `newBunch`. Luego, agregamos tanto el `chosenTransitions` como el `newBunch` a Π_t y a la cola de bunches por refinar. Ahora, tenemos dos bunches en Π_t que no son estables respecto a ambos bunches, por lo que debemos refinar los bloques de Π_s para que lo sean. Para esto, se utiliza la función `findSplittableBlocks`, donde se buscan todos los bloques que contienen transiciones en los dos nuevos bunches. Por cada bloque B encontrado, se crea un splitter primario y otro secundario y se guardan en una lista `splitterList`. El splitter primario contiene todas las transiciones de `chosenTransitions` que salen de B , y el splitter secundario contiene todas las transiciones de `newBunch` que salen de B . A su vez, se debe marcar como transiciones necesarias a reestabilizar todas las transiciones de `chosenTransitions`, es decir, las marcas del splitter primario. En esta primera versión, se utiliza un mapa `Map<Splitter, ↪ Set<Triple<Long, String, Long>>> markings` para almacenar estas marcas, asociando a cada splitter el conjunto de transiciones que fueron marcadas para la reestabilización. Por cada estado que es origen de una transición primaria, debemos identificar si ese estado también tiene una transición secundaria, y marcar a una transición secundaria por cada estado, añadiéndola a `markings`, asociadas al splitter secundario.

Luego, se itera sobre la `splitterList`. Por cada bloque B que se está observando para realizar la partición, se define el conjunto de estados R que son los estados de B que pueden alcanzar inertemente una transición del splitter y el conjunto de estados U que son los que no. Esta división en nuevos bloques R y U se hace a través de una función "split" que se detallará más adelante. Siguiendo la lógica del paper, si el splitter actual era primario, entonces luego del split, el bloque U es estable respecto al bunch `chosenTransitions`, por lo que se debe crear un nuevo splitter secundario para el bloque B , que solo contiene las transiciones de `chosenTransitions` que salen de R . Es decir, se reemplaza el splitter secundario de B por el nuevo splitter secundario para R . Al realizar el split de R y U , las transiciones inertes salientes de R hacia U , antes eran inertes pues estaban dentro del bloque U , pero ahora debemos identificarlas como no-inertes, ya que ahora cruzan dos bloques separados. Para realizar esta identificación, se utiliza la función `findNewNonInertTransitions`. Estas nuevas transiciones no-inertes, conforman un nuevo bunch que se debe agregar a Π_t . A su vez, tal como se menciona en el paper, R puede haberse vuelto inestable respecto al nuevo bunch creado. Por ello, se vuelve a utilizar la función `split` para volver a dividir R en dos bloques, N y R' . Si N y R' no son vacíos, entonces se reemplaza a R de Π_s por N y R' .

Acorde con el análisis hecho en el paper, la estabilidad no se preserva cuando aparecen nuevos estados bottom, por lo que debemos estabilizar al nuevo bloque N con respecto a todos los bunches a donde N tenga transiciones salientes. Para ello, se identifican todos los bottom states de N y se itera sobre Π_t para encontrar todos los bunches que contienen alguna transición saliente de N . Por cada bunch encontrado, se crea un splitter secundario. Por último, se itera sobre los estados bottom identificados de N y se marca a una transición

por cada estado. Estas marcas se realizan agregando a la primera transición del estado en una lista de estados `marksForRestabilization`.

Retomando, la función `split` es el corazón del algoritmo. La función `split` es la que se encarga de dividir al bloque B en los bloques R y U , y la implementación de esta función es la que hace la diferencia en el paper para poder alcanzar la cota de la complejidad. En el paper se detalla a la función `split` como dos corutinas que se ejecutan en paralelo. Sin embargo, con vistas a nuestro objetivo de poder realizar primero una primera implementación del algoritmo y validar su correctitud, en esta primera versión no implementamos a la función como dos corutinas. La función `split` tiene la siguiente firma:

```
private static Pair<Set<Long>, Set<Long>> split(
    Set<Long> B,
    Set<Triple<Long, String, Long>> transitions,
    Set<Triple<Long, String, Long>> currentMarks,
    MTS<Long, String> toMinimise,
    Set<String> tauLabels,
    Map<Long, Set<Long>> stateToSCCMap)
```

Es decir, la función `split` recibe como argumentos el bloque B a dividir, las transiciones del splitter asociado, las marcas del splitter asociado, el LTS a minimizar, el conjunto de etiquetas identificadas como tau y el mapa de estados a bloques. A su vez, `split` devuelve un par de conjuntos de estados, el bloque R y el bloque U . En esta versión, `split` se limita a calcular el conjunto R usando backward reachability para las transiciones inertes. Para comenzar este procesamiento, todos los estados con alguna transición marcada (es decir, incluida en `currentMarks`), son puestos en el nuevo bloque R . A su vez, se recorren todas las transiciones del splitter para incluir también en R a los estados desde los cuales estas transiciones salen. Luego, se construye el mapa `inertPredecessorsInB`, que almacena para cada estado cuáles son sus predecesores inertes. Luego, se propagan las transiciones inertes hacia atrás, utilizando una estructura `worklistR` que comienza con todos los estados de R , y por cada uno de ellos busca qué estados son predecesores inertes y los agrega a R y a la `worklist` para procesarlos también. El proceso continúa hasta que no queden predecesores inertes y por lo tanto, R sea estable. Luego, se devuelve el nuevo bloque R y el nuevo bloque U , definido como el complemento de R en B .

5.3. Primeros tests

Como se mencionó anteriormente, la estrategia fue desarrollar una primera versión fiel al algoritmo para entender la lógica y poder validar su correctitud. Por eso, luego de realizar esta primera implementación, se desarrollaron casos de tests para verificar los funcionamientos básicos del algoritmo. Se realizaron en primera instancia tests unitarios básicos, con casos desde 2 estados hasta 80 estados. Se creó un archivo de testeo unitario llamado `BranchingEquivalenceTest.java`, ubicado en `mtstools/DCS/Compositional/Tests/`, donde se implementó la lógica básica para a raíz de archivos `.lts` llamar al algoritmo de Branching Equivalence y luego del algoritmo de Weak Equivalence y comparar el grafo resultante obtenido. Para esto, primero hicimos pruebas de todos los casos donde ambos algoritmos tenían que reducir al mismo gráfico, es decir casos donde los algoritmos se comporten de forma equivalente. Luego de validar estos casos, nuestro objetivo fue encontrar un caso donde la lógica de los algoritmos difiera, es decir que por el caso de estudio del grafo elegido, ambas reducciones por la definición de equivalencia que utilizan, se comporten de forma diferente. Para ello, se implementó el caso de test presentado por ROB J. VAN GLAB-

BEEK y W. PETER WEIJLAND en <https://dl.acm.org/doi/epdf/10.1145/233551.233556>. Este caso de test fue importante para poder asegurarnos de que el comportamiento de Branching Equivalence realmente se comporte como esperábamos, y nos permitió validar de forma más robusta su implementación. A su vez, nos permitió desarrollar a un nivel más práctico las diferencias reales entre ambos algoritmos.

5.4. Optimizaciones y mejoras (v1)

5.4.1. Estructura del bucle: dos fases que alternan

Para la primer version del algoritmo con optimizaciones, la mayor diferencia con la estructura del paper es que se implementan dos fases que alternan al mismo nivel en lugar de anidar un while externo sobre bunches con un for interno sobre splitters.

- 5.4.2. Fase 1: estabilización de estados
- 5.4.3. Fase 2: refinamiento de bunches
- 5.4.4. Construcción del MTS minimizado
- 5.5. Decisiones de diseño no dictadas por el paper
 - 5.5.1. Dos fases en lugar del nesting outer-while + inner-for
 - 5.5.2. Identidad por referencia para la cola de bunches
 - 5.5.3. Índice inverso estado destino $\rightarrow$ bunches
 - 5.5.4. Subdivisión sistemática de splitters obsoletos
 - 5.5.5. SCCs como unidad atómica en split
 - 5.5.6. Detección de bottom states por SCC
 - 5.5.7. Hashing de bloques en tiempo constante
 - 5.5.8. Restabilización en ambas direcciones
- 5.6. Adaptaciones al contexto composicional
 - 5.6.1. Estado de error como bloque inicial separado
 - 5.6.2. Self-loops τ controlables
 - 5.6.3. Initiating actions de fluents fuera de τ
 - 5.6.4. Reuso de partición precomputada
 - 5.6.5. Eventos controlables, no-controlables y estados marcados
- 5.7. Validación de correctitud
 - 5.7.1. Estrategia
 - 5.7.2. Tests automatizados
 - 5.7.3. Casos de borde cubiertos

6. CONCLUSIONES

A lo largo de este trabajo buscamos analizar la forma en la que el algoritmo DCS permite ser conceptualizado en una serie de *fases*, de las cuales comprendimos su funcionamiento durante la ejecución del algoritmo general de forma agnóstica a la heurística utilizada. Mediante la identificación de estas fases pudimos ver que el algoritmo (DCS) atraviesa etapas cuyos objetivos parciales no siempre se mantienen, más allá de ser etapas que conducen a un objetivo en común (resolver el problema de control en general). Por ejemplo, para encontrar una solución, primero es necesario encontrar un estado marcado y luego encontrar la manera de llegar de forma controlable a él.

Para llegar a ese análisis, primero atravesamos un proceso exploratorio sobre el algoritmo estándar y parte de su implementación para finalmente acceder al algoritmo *on-the-fly* y sus detalles. Este último, sumado a los detalles de la *heurística RA* fueron estudiados con precisión para poder desarrollar este trabajo.

Con este enfoque en mente y el trabajo realizado hemos logrado los siguientes objetivos:

- Definir y entender en profundidad las etapas que atraviesa el algoritmo (DCS) durante su ejecución y cuantificar el rendimiento de las heurísticas actuales para cada fase.
- Introducción de cambios a la heurística “Ready Abstraction” que impactaron en una de las fases de ejecución cuyo rendimiento es menor en términos relativos.
- Implementación de los cambios sobre la heurística incorporados en la herramienta MTSA [8]
- Mejora del rendimiento de la heurística con los cambios introducidos, tomando los casos del *benchmark* con ventana de mejora en la etapa 3.

Como trabajo a futuro, presentamos las siguientes ideas:

- Modificación del algoritmo de exploración *on-the-fly* para resolución de otros problemas, por ejemplo cambiando el objetivo de nonblocking por objetivos de tipo GR1.
- Análisis más profundo del impacto (positivo o negativo) que tienen los cambios propuestos en la exploración.
- Propuesta de nuevas modificaciones más refinadas a RA basándonos en el análisis de fases.

7. APÉNDICE

7.1. Medición del peso de cada Fase por heurística

7.1.1. BFS

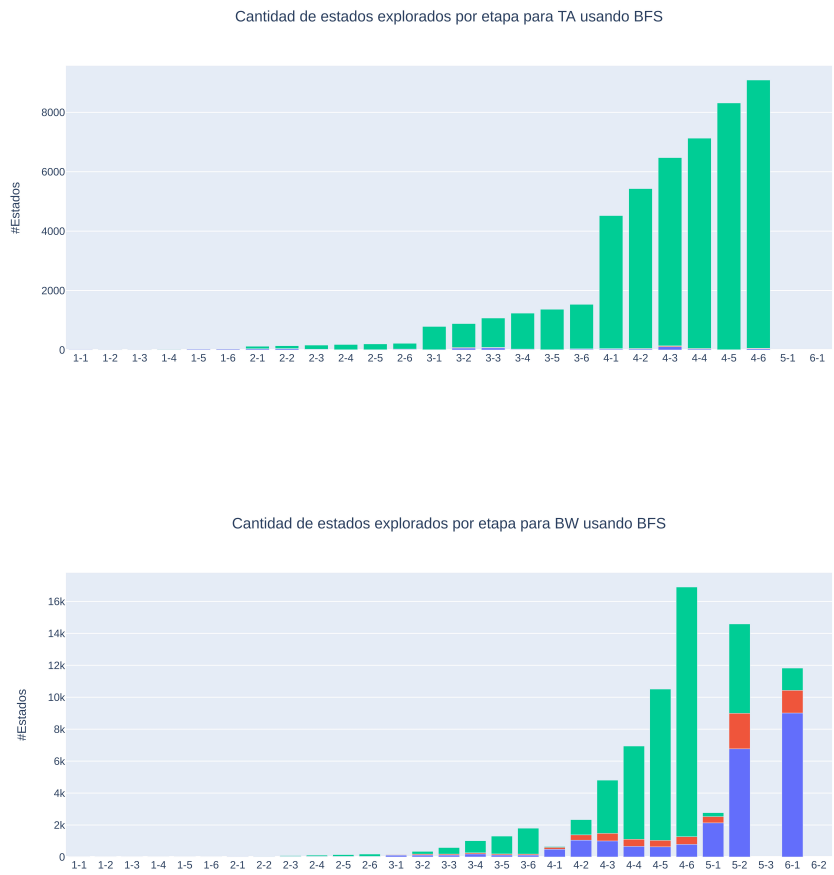


Fig. 7.1: Caption

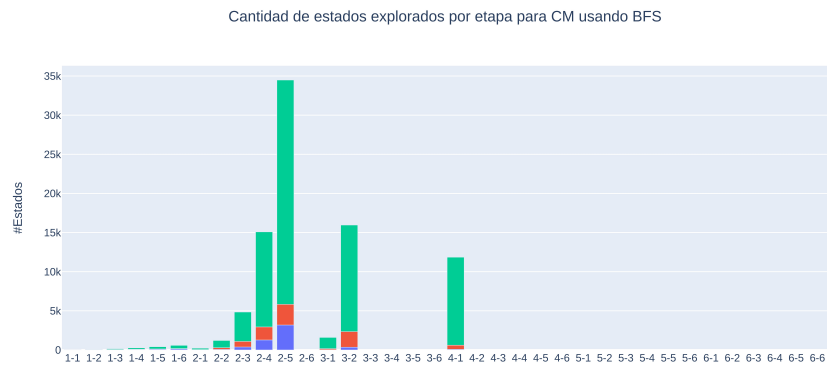


Fig. 7.2: Caption



Fig. 7.3: Caption



Fig. 7.4: Caption

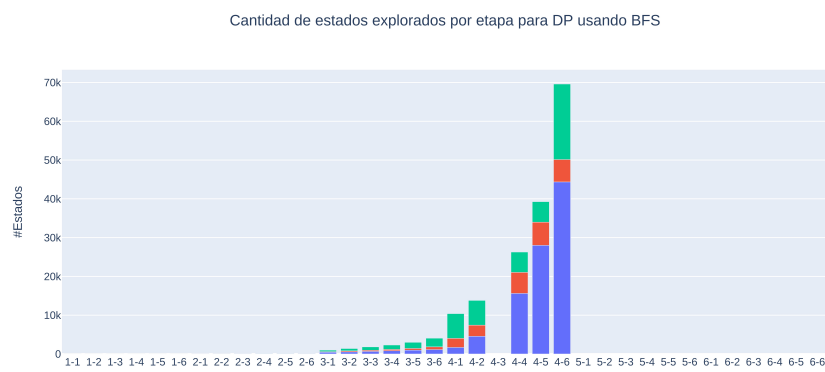


Fig. 7.5: Caption

7.1.2. RA

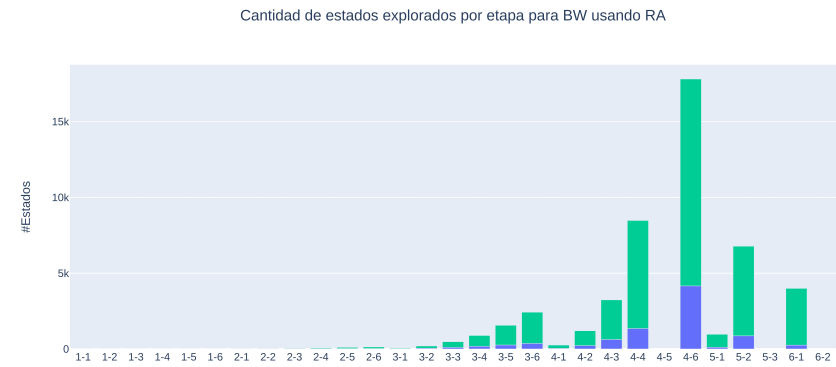


Fig. 7.6: Caption

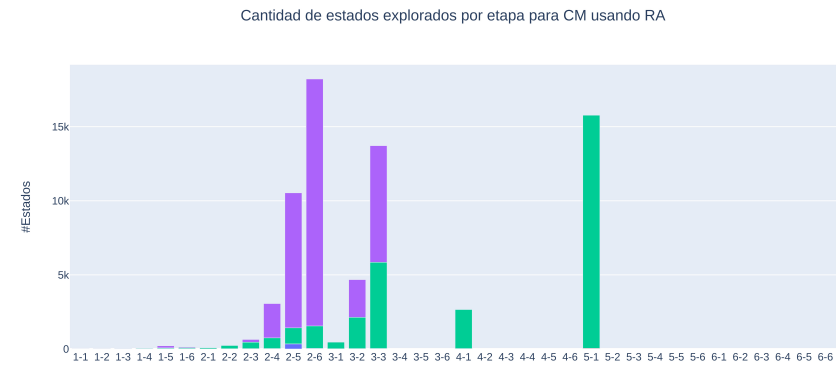


Fig. 7.7: Caption



Fig. 7.8: Caption



Fig. 7.9: Caption



Fig. 7.10: Caption

Bibliografía

- [1] D. Ciolek. Síntesis dirigida de controladores para sistemas de eventos discretos. PhD thesis, Laboratorio de Fundamentos y Herramientas para la Ingeniería del Software (LaFHIS), FCEyN, UBA ,2018.
- [2] M. Durán, F. Zanollo. Síntesis dirigida de controladores para requerimientos de tipo non-blocking. Tesis de Licenciatura, Laboratorio de Fundamentos y Herramientas para la Ingeniería del Software (LaFHIS), FCEyN, UBA, 2021.
- [3] Julian Braier, Daniel Ciolek, Matias Duran, Florencia Zanollo, Victor Braberman, Nicolas D’Ippolito, Sebastian Uchitel, Nicolas Pazos. *On-the-fly Informed Search of Non-blocking Directed Controller*.
- [4] S. A. Zudaire, M. Garrett, and S. Uchitel. Iterator-based temporal logic task planning. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pages 11472–11478, 2020.
- [5] P. J. Ramadge and W. M. Wonham. Supervisory Control of a Class of Discrete Event Processes. *SIAM Journal on Control and Optimization*, 25, 1987.
- [6] A. Pnueli and R. Rosner. On the Synthesis of a Reactive Module. In *Proc. of the Symp. on Principles of Programming Languages, POPL*, pages 179–190, 1989.
- [7] Dana Nau, Malik Ghallab, and Paolo Traverso. *Automated Planning: Theory & Practice*. Morgan Kaufmann Publishers Inc., 2004.
- [8] Modal Transition System Analyser (MTSA) <http://mtsa.dc.uba.ar/>